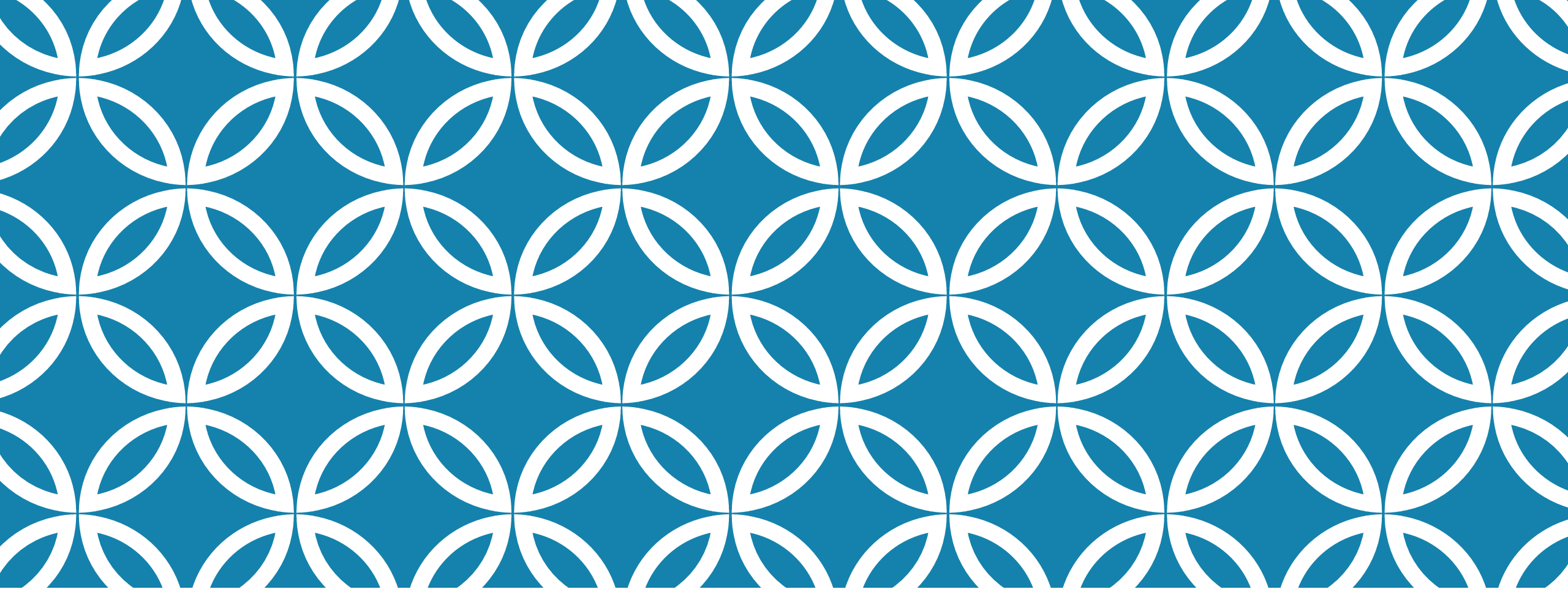




OBJECT ORIENTED PROGRAMMING

WEEK 2

29 AUG-03 SEP, 2022

Instructor:

Abdul Aziz
Assistant Professor
(School of Computing)
National University- FAST (KHI Campus)

ACKNOWLEDGMENT

- Publish material by Virtual University of Pakistan.
- Publish material by Deitel & Deitel.
- Publish material by Robert Lafore.

OBJECT

Technical Definition:

- “An Object is an Instance of a class”
- “An Object is the implementation of a class”

In general we say :

“ Any tangible thing for which we want to save Information”

Now onwards we treat object technically.

SOME EXAMPLES

SHOES



Men sandal



Female sandal



Loafers



Pumpies

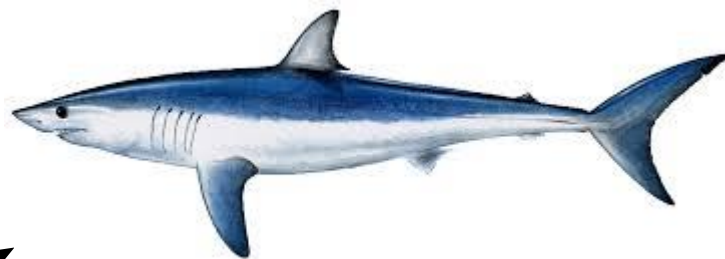


Formal



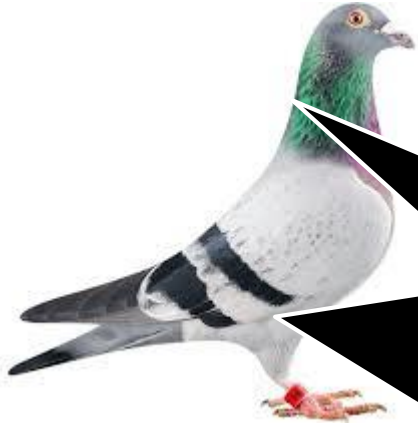
Humans





FISH





BIRD



DISCUSSION

- Object belongs to a group.
- Which similar.
- Have some common attributes.
- Have some common behaviors.
- We can categorized objects on some basic features ?

CLASS

- Collection of Similar object.
- The objects that share some common features.
- It is the a design of an object.
- It is a detail of an object.
- It tell us what an object contains in it.

Technical Definition:

“ A class is blueprint of an object”

SUMMARIZE

A class :

- It's a blue print .
- It's a design or template.

An Object:

- Its an instance of a class.
- Implementation of a class.

NOTE: Classes are invisible, object are visible

GENERALIZED CLASS

- The class that only exhibits the common features of its objects.

Examples:

- ANIMAL
- BIRDS
- HUMAN
- No object of generalized class is found.

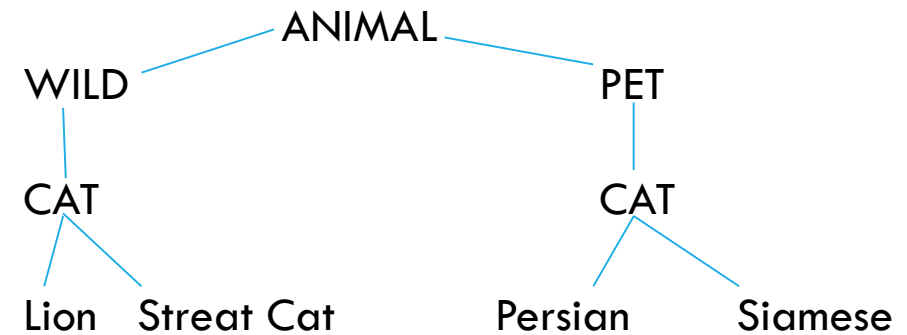
SPECIALIZED CLASS

- The class that exhibits different or unique features (behaviors)

ANIMAL (Generalized)

- Specialized:

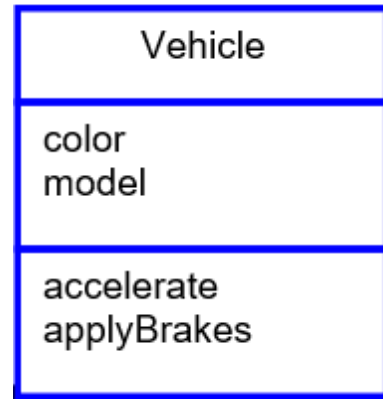
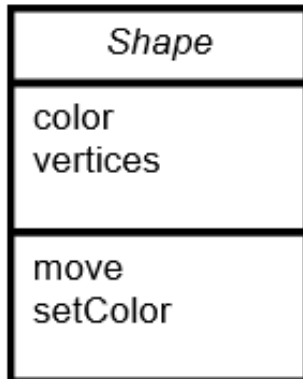
- Mammals
- Cats
- Dog



TYPE OF CLASSES

- 1- Abstract Class: The classes we make against abstract concepts are called abstract classes. Abstract Classes can not exist standalone.
- 2- Concrete Class: The entities that actually we see in our real world are called concrete objects and classes made against these objects are called concrete classes.
- 3- Sub-type: Sub-typing means that derived class is behaviorally compatible with the base class. Also known as Extension.
- 4- Specialized class: Specialization means that derived class is behaviorally incompatible with the base class

ABSTRACT CLASS



CONCRETE CLASS

Line	Circle	Triangle
color vertices length	color vertices radius	color vertices angle
move setColor getLength	move setColor computeArea	move setColor computeArea

Line is shape

Circle is a shape

Triangle is a shape

ENCAPSULATION

Encapsulation is the process of hiding an object's implementation from another object, while presenting only the interfaces that should be visible.

PRINCIPLES OF ENCAPSULATION

“Don’t ask how I do it, but this is what I can do”

- The encapsulated object

“I don’t care how, just do your job, and I’ll do mine”

- One encapsulated object to another

ENCAPSULATING A CLASS

- Members of a class must always be declared with the minimum level of visibility.
- Provide *setters and getters* (also known as accessors/mutators) to allow *controlled* access to private data.
- Provide other public methods (known as *interfaces*) that other objects must adhere to in order to interact with the object.

ACCESS MODIFIERS

private

Private features of the Sample class can only be accessed from within the class itself.

protected

Classes that are in the package and all its subclasses may access protected features of the Sample class.

public

All classes may access public features of the Sample class.

SYNTAX

```
class temp {  
    private:  
        .....  
    protected:  
        .....  
    public:  
        .....  
};
```

EXAMPLE

```
class temp{  
    private:  
        int a;  
        float f1;  
    public:  
        void func1(void){  
        }  
  
        void func2(void){  
        }  
};
```